



DATA STRUCTURE STACK

In this semester, we'll learn some of data structures algorithms

- 1) Stack
 - 2) Queue
 - 3) Linked lists
 - 4) Doubly Linked lists
 - 5) Linked Stack
 - 6) Linked Queue
 - 7) Tree
-

Stack

At first, what's the meaning of stack ?

Stack is Ordered List of Elements of Same Type

بمعنى أنه قائمة بعناصر من نفس النوع مرتبة بشكل معين

How this elements sorted ?

It sorted in way that last element will be the first one when pop from stack

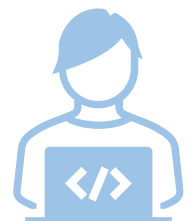
بمعنى ان آخر عنصر دخل ال stack هو أول واحد سيتم سحبه من ال stack

So , Stack is called “LIFO”

“Last In First Out”

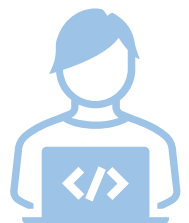
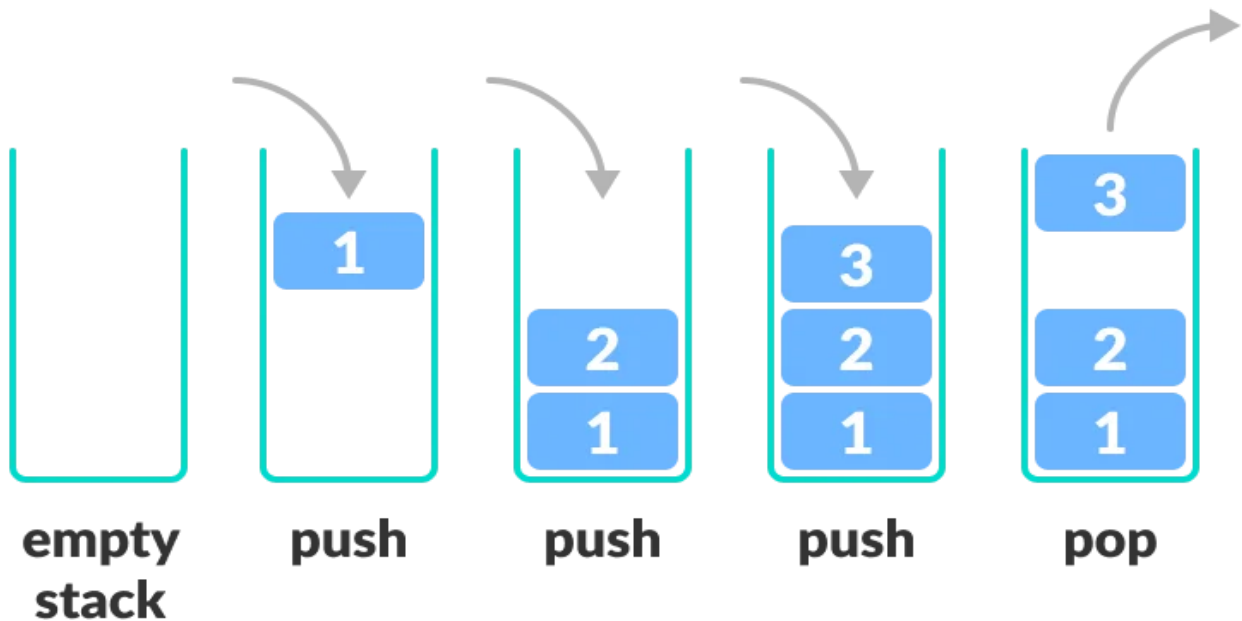
, also called “FILO”

“First In Last Out”



تعالى نفكر كده في شكل ال Stack و نكون فكره عنه أصلا

- تخيل ال stack انه كوب او صندوق مغلق من اسفله
- لما نيجي نحط مثلا كورة في الصندوق ده بتتنزل تحط خالص ونحط تانى فتنزل الكورة دى فوق الأولى وهكذا كل ما اضيف
- طب لما نيجي نسحب من الكوباية هل هنسحب اول واحدة خالص؟
- لا هنسحب الأخيرة ثم اللى قبلها وهكذا لذلك اخر كورة دخلت هي اول واحدة خرجت لما جيت اسحب
- بنسمى الإضافة لل Stack عملية ال **“push”**
- بنسمى السحب من ال Stack عملية ال **“pop”**



1) Characteristics of Stack

- Stack is Linear - non-primitive data structure.
 - Stack is Ordered List of Elements of Same Type.
-

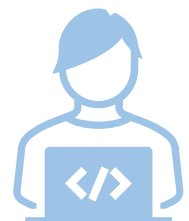
2) Some applications of stack

- Function call management (call stack)
- Expression evaluation and syntax parsing
- Backtracking algorithms
- Undo mechanisms in text editors

نسرش كويس على استخدامات ال Stack
بيجي عليها أسئلة

3) Operations (Functions) performed on Stack

- Create the stack, leaving it empty(initialize).
- Determine whether the stack is empty or not.
- Determine whether the stack is full or not.
- Push a new entry onto the top of the stack
- Pop the entry off the top of the stack.



Let's see stack in deeper level

قبل اى حاجة محتاجين نفهم ال parameters

Ex . `main{ func(x,y) } → func(*x,*y)`

كدا انا بعت لل فانكشن نسخة من `x` و `y` والفانكشن استلمت ال addresses بتاعة النسخة دي فبالتالى اى حاجة هعملها على `x` و `y` مش هتسمع في الأصليين لانى بعدل على النسخة

Ex. `main{ func(&x,&y) } → func(x,y)`

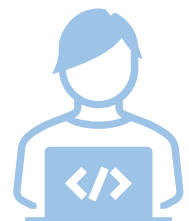
كده انا بعت لل فانكشن ال address بتاع `x` و `y` فبالتالى لو حبيت اعدل عالقيمة الاصلية ليهم هكتب جوا الفانكشن

`func(x,y){`

`*x =5;` كده انا بعدل على قيمة `x` الاصلية اللى جوا ال address

`*y=6;`

`}`



نخش بقى عال Stack Operations

1) Initialize Stack

In “stack.h” file

```
#define maxsize 10;
```

\\ control the number of max elements in stack

```
typedef char EntryType;
```

*stack elements type

*/ (عشان لو احتجت اغير النوع اغيره من هنا على طول)

```
typedef struct{
```

```
    int top;
```

* like Indicator indicate on last element in stack */

```
    EntryType entry[maxsize];
```

\\ like writing char entry[10]

```
} Stack;
```

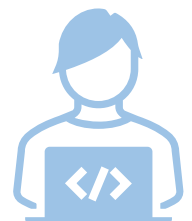
انا كده عرّفت ال stack كانه datatype زى int ,float,double ...etc

فلما اعرف stack جديد بكتب Stack s;

ساعتها ال s دى عبارة عن struct جواه top و entry[maxsize] فلو عايز احط قيمة ليهم مثلا

```
s.top=5;
```

```
s.entry[2]= 'a' ;
```



2) Creating stack (prepare stack to push & pop)

```
void create(stack *s){
    s->top=0;
}
```

هنا انا بشاور على ال top اللى جوا ال stack وبخليها ب 0
متنساش ان ال top بيعرفنى اخر عنصر فى ال stack فى index كام
بما انى لسه بعمله يعنى فاضى فبالتالى ال top ب 0

طب انا ليه بستانق ال stack address (stack *s) مش (stack s) ؟

عشان التغيرات اللى هتحصل تأثر على ال s الاصلية عشان لو بعت
(stack s) هيتبعت نسخة من ال s اللى انا عايز اعدل عليها فالاصلية مش
هتتأثر

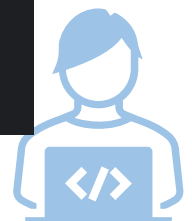
طب ال s->top = 0 دى معناها ايه ؟

معناها 0 = (s.top)*

3) Check if this stack is Empty or Full

```
bool isEmpty(stack *s){
    if(s->top==0) return true;
    else return false;
}

bool isFull(stack *s){
    if (s->top==maxsize) return true;
    else return false;
}
```



لو انا بسحب من ال stack لازم يكون مش فاضي و العكس لما بضيف ليه

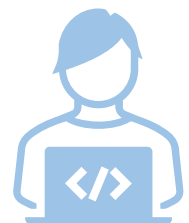
4) Clear stack

```
void clear(stack *s){  
    s->top=0;  
}
```

برجع ال top تانى بصفر

5) Size of stack

```
int size(stack *s){  
    return s->top;  
}
```



ركز أوى بقى في اللى جى.....

6) push (add element to stack)

```
void push(Stack *s, stackEntry e){
    if(s->top>=maxsize) return;
    s->arr[s->top]=e;
    s->top++;
}
```

1- ال parameters

ده عبارة عن ال address لل stack s $\rightarrow$ (Stack *s)

Entry e $\rightarrow$ ده العنصر اللى هضيفه لل stack

2- عملية ال push

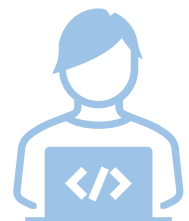
- أول حاجة بشوف ال stack مليان ولا لا من خلال ال function اللى عملناها isFull لو مليان هيطلع برا ال function و مش هيكمل

- $s->entry[++s->top] = e ;$
كأنى كتبت

$s->top++;$

$int Top=s->top;$

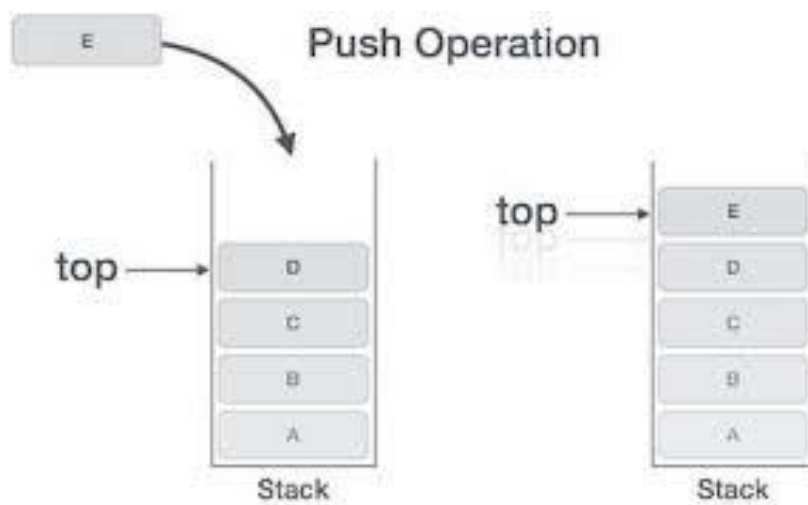
$s->entry[Top]=e;$



طب ايه اللي حصل في ال push ؟

مش المفروض ان ال top بيعرفنى ال index لآخر عنصر دخل ال stack ؟

لذلك هنا احنا زود 1 في ال top وحطيت فيه العنصر الجديد اللي انا ضفته



7) pop (pull elements that found in stack)

في ال pop هناك شكلين عشان اسحب بيهم ال element

1- من ال return

Ex.

```
entry popped_element;
```

```
Stack s;
```

```
create(&s);
```

```
push(&s,5);
```

```
popped_element=pop(&s);\\ (5) Stack عنصر في
```



2- انى احطها في address معين وابعته لل function

Ex.

```
entry *popped_element;
Stack s;
create(&s);
push(&s,5);
pop(&s, popped_element);
printf("%d",*popped_element ); \\ 5
```

نيجي بقى لل pop نفسها

1- بطريقة ال return

```
stackEntry pop3(Stack *s){
    if(s->top==0) return -1;
    return (s->arr[--s->top]);
}
```

2- بطريقة ال pointer variable

```
void pop2(Stack *s,stackEntry *e){
    if(s->top==0) return;
    *e=s->arr[--s->top];
}
```

ساعتها ال pointer الى انا بعته هيكون فيه اخر حاجة دخلت ال stack
ولما اعمل pop تانى هيطلعلى الى بعده وهكذا



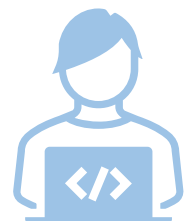
طب احنا دلوقتي عرفنا ازاي بنعمل ال stack طب ازاي بنجمع ال files
كلها في مكان واحد

(1) أول حاجة ملف stack.h

ده بنحط فيه ال installation و ال prototypes لل functions

bin	10/12/2023 6:33 PM	File folder	
obj	10/12/2023 6:33 PM	File folder	
global.h	11/11/2023 8:23 PM	Header file	1 KB
main.c	11/11/2023 8:49 PM	C source file	1 KB
main.exe	10/12/2023 7:03 PM	Application	54 KB
main.o	10/12/2023 7:03 PM	O File	2 KB
queue.c	11/11/2023 8:29 PM	C source file	1 KB
queue.h	11/11/2023 8:25 PM	Header file	1 KB
stack.file.cbp	11/11/2023 8:49 PM	project file	2 KB
stack.file.depend	11/11/2023 8:25 PM	DEPEND File	1 KB
stack.file.layout	11/11/2023 8:49 PM	LAYOUT File	2 KB
stack.c	10/31/2023 2:24 PM	C source file	1 KB
stack.h	11/11/2023 8:22 PM	Header file	1 KB
test.c	10/22/2023 9:24 AM	C source file	1 KB
test.o	10/22/2023 9:26 AM	O File	2 KB
testing.c	10/22/2023 9:21 AM	C source file	1 KB
testing.o	10/22/2023 9:21 AM	O File	2 KB

```
#ifndef STACKARRAY_STACK_H
#define STACKARRAY_STACK_H
#define maxsize 50
#define stackEntry char
typedef struct Stack{
    int top;
    stackEntry arr[maxsize];
}Stack;
void create(Stack *);
bool isEmpty(Stack *);
bool isFull(Stack *);
void push(Stack *,stackEntry );
void pop(Stack * ,stackEntry *);
int size(Stack *);
void clear(Stack *);
stackEntry firstElement(Stack *);
stackEntry lastElement(Stack );
void copyStack(Stack ,Stack *);
void displayStack(Stack);
stackEntry *traverse(Stack*);
#endif //STACKARRAY_STACK_H
```

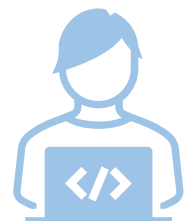


(2) بنعمل ملف تانى اسمه stack.c وبنحط جواه ال functions بيعمل include لل "stack.h"

bin	10/12/2023 6:33 PM	File folder	
obj	10/12/2023 6:33 PM	File folder	
global.h	11/11/2023 8:23 PM	Header file	1 KB
main.c	11/11/2023 8:49 PM	C source file	1 KB
main.exe	10/12/2023 7:03 PM	Application	54 KB
main.o	10/12/2023 7:03 PM	O File	2 KB
queue.c	11/11/2023 8:29 PM	C source file	1 KB
queue.h	11/11/2023 8:25 PM	Header file	1 KB
stack file.cbp	11/11/2023 8:49 PM	project file	2 KB
stack file.depend	11/11/2023 8:25 PM	DEPEND File	1 KB
stack file.layout	11/11/2023 8:49 PM	LAYOUT File	2 KB
stack.c	10/31/2023 2:24 PM	C source file	1 KB
stack.h	11/11/2023 8:22 PM	Header file	1 KB
test.c	10/22/2023 9:24 AM	C source file	1 KB
test.o	10/22/2023 9:26 AM	O File	2 KB
testing.c	10/22/2023 9:21 AM	C source file	1 KB
testing.o	10/22/2023 9:21 AM	O File	2 KB

```
#include "Stack.h"
#include "bits/stdc++.h"
void create(Stack *s){
    s->top=0;
}
bool isEmpty(Stack *s){
    if(s->top==0) return true;
    else return false;
}
bool isFull(Stack *s){
    if (s->top==maxsize) return true;
    else return false;
}
void push(Stack *s,stackEntry e){
    if(s->top>=maxsize) return;
    s->arr[s->top]=e;
    s->top++;
}
void push2(Stack *s,stackEntry e){
    if(s->top>=maxsize) return;
    s->arr[s->top++]=e;
}
void pop(Stack *s,stackEntry *e){
    if(s->top==0) return;
    s->top--;
    *e=s->arr[s->top];
}
```

```
void pop2(Stack *s,stackEntry *e){
    if(s->top==0) return;
    *e=s->arr[--s->top];
}
stackEntry pop3(Stack *s){
    if(s->top==0) return -1;
    return (s->arr[--s->top]);
}
void clear(Stack *s){
    s->top=0;
}
int size(Stack *s){
    if(s->top >= 0 && s->top <= maxsize)
        return s->top;
    return -1;
}
```



(3) بنيجي بعدها في ال main ونستعمل ال stack من خلال اننا نعمل
#include "stack.h"

```
Stack.h  Stack.cpp  main.cpp x
1 //
2 // Created by tefa on 7/28/2024.
3 //
4 #include "Stack.h"
5 #include "bits/stdc++.h"
6 int main(){
7     stack s;
8     create(&s);
9     int e,res;
10    printf(format: "enter an element to push it in the stack\n");
11    scanf( format: " %d",&e);
12    push(&s,e);
13    pop(&s,&res);
14    printf(format: "the last element pushed in the stack is %d",res);
15 }
16
17
18
19
```

Run - StackArray

Run StackArray x

D:\Raiders\DS\StackArray\cmake-build-debug\StackArray.exe

enter an element to push it in the stack

35

the last element pushed in the stack is 35

Process finished with exit code 0

